

Name:A.Shashidhar H.No:2303A51798 Batch:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																			
Program Name:B. Tech		Assignment Type: Lab	Academic Year:2025-2026																		
Course Coordinator Name		Dr. Rishabh Mittal																			
Instructor(s)Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> <tr><td>Mr. G. Kranthi</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju	Mr. G. Kranthi
Mr. S Naresh Kumar																					
Ms. B. Swathi																					
Dr. Sasanko Shekhar Gantayat																					
Mr. Md Sallauddin																					
Dr. Mathivanan																					
Mr. Y Srikanth																					
Ms. N Shilpa																					
Dr. Rishabh Mittal (Coordinator)																					
Dr. R. Prashant Kumar																					
Mr. Ankushavali MD																					
Mr. B Viswanath																					
Ms. Sujitha Reddy																					
Ms. A. Anitha																					
Ms. M.Madhuri																					
Ms. Katherashala Swetha																					
Ms. Velpula sumalatha																					
Mr. Bingi Raju																					
Mr. G. Kranthi																					
Course Code	23CS002PC304	Course Title	AI Assisted Coding																		
Year/Sem	III/I	Regulation	R23																		
Date and Day of Assignment	Week 9 - Thursday	Time(s)	23CSBTB01 To 23CSBTB52																		
Duration	2 Hours	Applicable to Batches	All Batches																		
AssignmentNumber:18.4 (Present assignment number)/24(Total number of assignments)																					
Q.No.	Question	ExpectedTime to complete																			
1	Lab 18: API Integration: Connecting to External Services with Error	Week 9																			

	Handling Lab Objectives • Learn how to integrate Python programs with external REST APIs. • Understand API request/response handling using requests or similar libraries. • Implement proper error handling for failed API calls (timeouts, invalid responses, rate limits). • Practice extracting and displaying meaningful information from API responses. Learning Outcome After completing this assignment, students will be able to: • Integrate external REST APIs using AI-assisted Python scripting. • Implement structured error handling for API failures and invalid inputs. • Safely parse and validate JSON responses from external services. • Develop resilient applications capable of handling real-world API limitations.	
	Task 1 – Movie Database API Integration Scenario You are building a movie recommendation tool that fetches movie details from a public Movie Database API. Task Use AI to generate a Python script that connects to a Movie API and retrieves movie information by title. Instructions • Use <code>requests</code> library to fetch movie details. • Accept movie title as user input. • Display: ◦ Title ◦ Release year ◦ Rating ◦ Plot summary • Implement error handling for: ◦ Invalid movie name	

- API key errors
- Network timeout
- Empty API response

Expected Output

- A working Python script that fetches movie details.
- Graceful error messages instead of crashes.
- Clean, readable output format.

The screenshot displays a development environment with three main components:

- Editor (Left):** A Python script named `AAC A 18.4.py` that defines a function `fetch_movie_details` to retrieve movie information from the OMDb API. The script includes error handling for timeouts, connection errors, and invalid API keys.
- Terminal (Bottom):** Shows the command prompt output where the script is executed. It prompts for an OMDb API key (26e990ca), a movie title (Dune), and displays the resulting JSON data for the movie "Dune".
- Sidebar (Right):** The Claude Code interface shows a "MOVIE RECOMMENDATION TOOL API INTEGRATION SCRIPT" patch. It explains that the script is updated to handle 401 responses by parsing the error payload into a specific API key error message. It also provides instructions on how to handle an invalid API key, such as re-opening the OMDb activation email.

```
AAC A 18.4.py 1 X
C:\Users\shash> student_conversion > AAC A 18.4.py > ...
6 def fetch_movie_details(title, api_key):
22     data = response.json()
23 except ValueError:
24     return None, "Invalid API response: Could not parse server response."
25
26 if response.status_code == 401:
27     error_message = data.get("Error", "Unauthorized API access.") if isinstance(data, dict) else "Unauth
28     return None, f"API key error: {error_message}"
29
30 if response.status_code >= 400:
31     error_message = data.get("Error", "Unknown API error.") if isinstance(data, dict) else "Unknown API error."
32     return None, f"API error: {error_message}"
33
34 if not data:
35     return None, "Empty API response: No data found."
36
37 if data.get("Response") == "False":
38     error_message = data.get("Error", "Unknown API error.")
39     lowered = error_message.lower()
40
41 if "api key" in lowered:
42     return None, f"API key error: {error_message}"
43 if "movie not found" in lowered:
44     return None, "Invalid movie name: No movie found with that title."
45
46 title_value = data.get("Title", "N/A")
47 year_value = data.get("Year", "N/A")
48 rating_value = data.get("imdbRating", "N/A")
49 plot_value = data.get("Plot", "N/A")
50
51 if all(v in ("", "N/A", None) for v in [title_value, year_value, rating_value, plot_value]):
52     return None, "Empty API response: Movie data is missing."
53
54 return {
55     "Title": title_value,
56     "Release Year": year_value,
57     "Rating": rating_value,
58     "Plot Summary": plot_value
59 }, None
60
61
62 def main():
63     title = input("Enter movie title: ")
64     api_key = input("Enter your OMDb API key: ")
65     fetch_movie_details(title, api_key)
66
67 if __name__ == '__main__':
68     main()
```

PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58016' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter your OMDb API key: 26ee90ca

HTTP error: 401 Client Error: Unauthorized for url: https://www.omdbapi.com/?t=Dune&apikey=26ee90ca

PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52859' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter your OMDb API key: 26ee90ca

Enter movie title: Dune

API key error: Invalid API key!

```
AAC A 18.4.py 1 X
C:\Users\shash> student_conversion > AAC A 18.4.py > ...
41 if "api key" in lowered:
42     return None, f"API key error: {error_message}"
43 if "movie not found" in lowered:
44     return None, "Invalid movie name: No movie found with that title."
45 return None, f"API error: {error_message}"
46
47 title_value = data.get("Title", "N/A")
48 year_value = data.get("Year", "N/A")
49 rating_value = data.get("imdbRating", "N/A")
50 plot_value = data.get("Plot", "N/A")
51
52 if all(v in ("", "N/A", None) for v in [title_value, year_value, rating_value, plot_value]):
53     return None, "Empty API response: Movie data is missing."
54
55 return {
56     "Title": title_value,
57     "Release Year": year_value,
58     "Rating": rating_value,
59     "Plot Summary": plot_value
60 }, None
61
62 def main():
63     title = input("Enter movie title: ")
64     api_key = input("Enter your OMDb API key: ")
65     fetch_movie_details(title, api_key)
66
67 if __name__ == '__main__':
68     main()
```

PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58016' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter your OMDb API key: 26ee90ca

HTTP error: 401 Client Error: Unauthorized for url: https://www.omdbapi.com/?t=Dune&apikey=26ee90ca

PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52859' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter your OMDb API key: 26ee90ca

Enter movie title: Dune

API key error: Invalid API key!

```
AAC A 18.4.py 1 X
C: > Users > shash > student_conversion > AAC A 18.4.py > ...
62 def main():
69     return
70
71     movie_title = input("Enter movie title: ").strip()
72     if not movie_title:
73         print("Invalid movie name: Title cannot be empty.")
74         return
75
76     movie, error = fetch_movie_details(movie_title, api_key)
77
78     if error:
79         print(error)
80         return
81
82     print("\nMovie Details")
83     print("-" * 40)
84     print(f"Title      : {movie['Title']}")
85     print(f"Release Year : {movie['Release Year']}")
86     print(f"Rating     : {movie['Rating']}")
87     print(f"Plot Summary : {movie['Plot Summary']}")
88
89 if __name__ == "__main__":
90     main()
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion\ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.12.0\bin\python.exe' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter your OMDb API key: 26ee90ca

HTTP error: 401 Client Error: Unauthorized for url: https://www.omdbapi.com/?t=dune&apikey=26ee90ca

● PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion\ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.12.0\bin\python.exe' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter your OMDb API key: 26ee90ca

Enter movie title: Dune

API key error: Invalid API key!

Task 2 – Public Transport Schedule API

Scenario

A city transport app needs to fetch bus/train arrival timings from a transport API.

Task

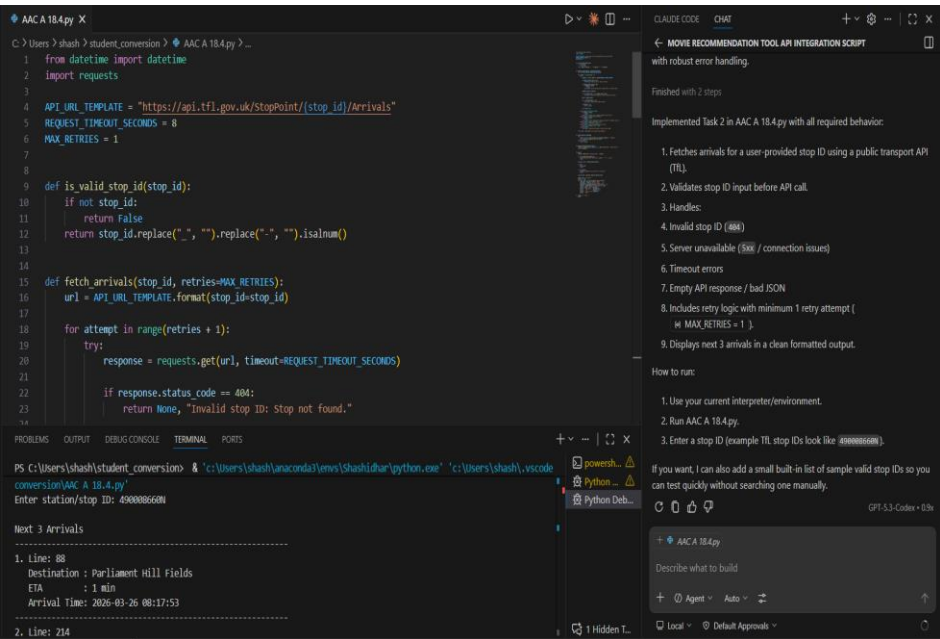
Use AI assistance to integrate a transport schedule API.

Instructions

- Fetch arrival times for a given station/stop ID.
- Validate user input.
- Handle errors such as:
 - Invalid stop ID
 - Server unavailable
 - Timeout errors
- Display next 3 arrival times in formatted output.
- Implement retry logic (minimum 1 retry attempt).

Expected Output

- Python script showing upcoming arrivals.
- Retry mechanism working properly.
- Proper exception handling using try-except blocks.



```
AAC A 18.4.py X
C:\Users\shash> student_conversion > AAC A 18.4.py > ...
15 def fetch_arrivals(stop_id, retries=MAX_RETRIES):
24     if response.status_code == 404:
25         return None, "Invalid stop ID: Stop not found."
26
27     if response.status_code >= 500:
28         if attempt < retries:
29             continue
30         return None, "Server unavailable: Transport API is currently down."
31
32     response.raise_for_status()
33
34     if not response.text or not response.text.strip():
35         return None, "Empty API response: No data received from server."
36
37     data = response.json()
38
39     if not isinstance(data, list):
40         return None, "Invalid API response format."
41
42     if len(data) == 0:
43         return [], None
44
45     return data, None

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\shash\student_conversion> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode
2. Line: 214
Destination : Highgate Village
ETA : 6 min
Arrival Time: 2026-03-26 08:22:30
-----
3. Line: 88
Destination : Parliament Hill Fields
ETA : 12 min
Arrival Time: 2026-03-26 08:29:00
-----
1 Hidden T...
```

```
AAC A 18.4.py X
C:\Users\shash> student_conversion > AAC A 18.4.py > ...
15 def fetch_arrivals(stop_id, retries=MAX_RETRIES):
44
45     except requests.exceptions.Timeout:
46         if attempt < retries:
47             continue
48         return None, "Timeout error: Request timed out after retry."
49     except requests.exceptions.ConnectionError:
50         if attempt < retries:
51             continue
52         return None, "Server unavailable: Could not connect to transport service."
53     except requests.exceptions.HTTPError as err:
54         return None, f"HTTP error: {err}"
55     except requests.exceptions.RequestException as err:
56         return None, f"Request error: {err}"
57     except ValueError:
58         return None, "Invalid API response: Could not parse JSON."
59
60     return None, "Unexpected error during retry handling."
61
62
63 def format_time(iso_timestamp):
64     try:
65         parsed = datetime.fromisoformat(iso_timestamp.replace("Z", "+00:00"))
66         return parsed.strftime("%Y-%m-%d %H:%M:%S")
67
68
69 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\shash\student_conversion> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode
2. Line: 214
Destination : Highgate Village
ETA : 6 min
Arrival Time: 2026-03-26 08:22:30
-----
3. Line: 88
Destination : Parliament Hill Fields
ETA : 12 min
Arrival Time: 2026-03-26 08:29:00
-----
1 Hidden T...
```

AAC A 18.4.py X

C:\Users\shash> student_conversion > AAC A 18.4.py > ...

```
63 def format_time(iso_timestamp):
64     return "Unknown"
65
66
67
68
69
70
71 def get_next_three_arrivals(arrivals):
72     ordered = sorted(arrivals, key=lambda x: x.get("timeToStation", float("inf")))
73     return ordered[:3]
74
75
76
77 def main():
78     stop_id = input("Enter station/stop ID: ").strip()
79
80     if not is_valid_stop_id(stop_id):
81         print("Invalid stop ID: Use letters, numbers, '-' or '_' only.")
82         return
83
84     arrivals, error = fetch_arrivals(stop_id)
85
86     if error:
87         print(error)
88         return
89
90     if not arrivals:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\student_conversion> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode

```
2. Line: 214
  Destination : Highgate Village
  ETA         : 6 min
  Arrival Time: 2026-03-26 08:22:30
-----
```

```
3. Line: 88
  Destination : Parliament Hill Fields
  ETA         : 12 min
  Arrival Time: 2026-03-26 08:29:00
-----
```

AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > ...

```
76 def main():
89     if not arrivals:
90         print("No upcoming arrivals found for this stop.")
91         return
92
93     next_arrivals = get_next_three_arrivals(arrivals)
94
95     print("\nNext 3 Arrivals")
96     print("-" * 60)
97     for index, item in enumerate(next_arrivals, start=1):
98         line_name = item.get("lineName", "N/A")
99         destination = item.get("destinationName", "N/A")
100         expected = format_time(item.get("expectedArrival", ""))
101         minutes = round(item.get("timeToStation", 0) / 60)
102         print(f"{index}. Line: {line_name}")
103         print(f"    Destination : {destination}")
104         print(f"    ETA           : {minutes} min")
105         print(f"    Arrival Time: {expected}")
106         print("-" * 60)
107
108 if __name__ == "__main__":
109     main()
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\shash\student_conversion> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python

2. Line: 214

Destination : Highgate Village

ETA : 6 min

Arrival Time: 2026-03-26 08:22:30

3. Line: 88

Destination : Parliament Hill Fields

ETA : 12 min

Arrival Time: 2026-03-26 08:29:00

Task 3 – Stock Market Data API

Scenario

You are developing a finance dashboard that retrieves live stock prices.

Task

Use AI to generate a Python script that fetches stock data from a financial API.

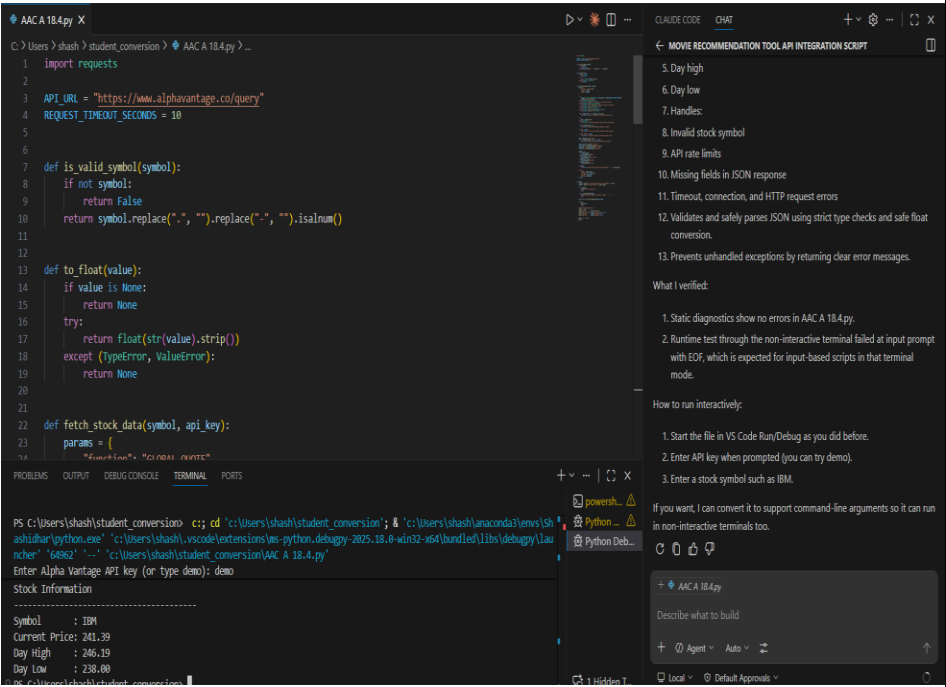
Instructions

- Accept stock symbol as input.

- Retrieve:
 - Current price
 - Day high
 - Day low
- Handle:
 - Invalid stock symbol
 - API rate limits
 - Missing fields in JSON response
- Validate and safely parse JSON response.

Expected Output

- Script fetches and displays stock information.
- No unhandled exceptions.
- Clear error messages for invalid symbols or rate limits.



AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > ...

```
22 def fetch_stock_data(symbol, api_key):
23     """Fetch stock data from Alpha Vantage API"""
24     symbol = symbol.upper()
25     "symbol": symbol,
26     "apikey": api_key,
27 }
28
29 try:
30     response = requests.get(API_URL, params=params, timeout=REQUEST_TIMEOUT_SECONDS)
31     response.raise_for_status()
32 except requests.exceptions.Timeout:
33     return None, "Timeout error: Request to stock API timed out."
34 except requests.exceptions.ConnectionError:
35     return None, "Server unavailable: Could not connect to stock API."
36 except requests.exceptions.HTTPError as err:
37     return None, f"HTTP error: {err}"
38 except requests.exceptions.RequestException as err:
39     return None, f"Request error: {err}"
40
41 if not response.text or not response.text.strip():
42     return None, "Empty API response: No data received from server."
43
44 try:
45     data = response.json()
46 except ValueError:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anac
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib
ncher' '64962' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
```

Enter Alpha Vantage API key (or type demo): demo

Stock Information

```
-----
Symbol      : IBM
Current Price: 241.39
Day High    : 246.19
Day Low     : 238.00
```

PS C:\Users\shash\student_conversion> |

AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > ...

```
22 def fetch_stock_data(symbol, api_key):
46     except ValueError:
47         return None, "Invalid JSON response: Could not parse API response."
48
49     if not isinstance(data, dict):
50         return None, "Invalid JSON response: Expected an object."
51
52     if "Note" in data:
53         return None, "API rate limit reached: Please wait and try again."
54
55     if "Error Message" in data:
56         return None, "Invalid stock symbol: No matching symbol found."
57
58     quote = data.get("Global Quote")
59     if not isinstance(quote, dict) or not quote:
60         return None, "Invalid stock symbol: No quote data returned."
61
62     symbol_value = quote.get("01. symbol")
63     price_value = to_float(quote.get("05. price"))
64     high_value = to_float(quote.get("03. high"))
65     low_value = to_float(quote.get("04. low"))
66
67     missing = []
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\U
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-
ncher' '64962' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
```

Enter Alpha Vantage API key (or type demo): demo

Stock Information

```
-----
Symbol      : IBM
Current Price: 241.39
Day High    : 246.19
Day Low     : 238.00
```

© PS C:\Users\shash\student_conversion>

```
AAC A 18.4.py X
C: > Users > shash > student_conversion > AAC A 18.4.py > ...
88 def main():
92     if not api_key:
93         print("API key is required.")
94         return
95
96     if not is_valid_symbol(symbol):
97         print("Invalid stock symbol: Use letters, numbers, '.', or '-'.")
98         return
99
100    stock, error = fetch_stock_data(symbol, api_key)
101
102    if error:
103        print(error)
104        return
105
106    print("\nStock Information")
107    print("-" * 40)
108    print(f"Symbol      : {stock['symbol']}")
109    print(f"Current Price: {stock['current_price']:.2f}")
110    print(f"Day High     : {stock['day_high']:.2f}")
111    print(f"Day Low      : {stock['day_low']:.2f}")
112
113    if __name__ == "__main__":
114        main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion'; & 'c:\Us
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x
ncher' '64962' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter Alpha Vantage API key (or type demo): demo
Stock Information
-----
Symbol      : IBM
Current Price: 241.39
Day High     : 246.19
Day Low      : 238.00
PS C:\Users\shash\student_conversion> |
```

Task 4 – Geolocation API (IP Address Lookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user based on IP address.

Task

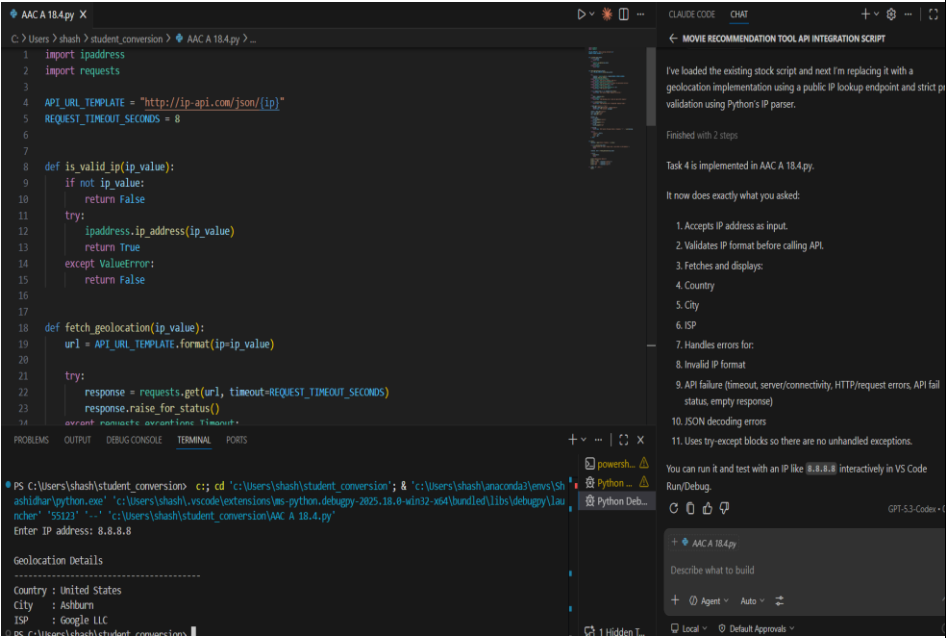
Integrate a Geolocation API using AI assistance.

Instructions

- Accept IP address as input.
- Fetch:
 - Country
 - City
 - ISP
- Validate IP format before making API call.
- Handle:
 - Invalid IP format
 - API failure
 - JSON decoding errors

Expected Output

- A script that returns geolocation details.
- Input validation implemented.
- Robust error handling included.



```
C:\> Users > shash > student_conversion > AAC A 18.4.py > ...
18 def fetch_geolocation(ip_value):
19     ...
24 except requests.exceptions.Timeout:
25     return None, "API failure: Request timed out."
26 except requests.exceptions.ConnectionError:
27     return None, "API failure: Could not connect to geolocation service."
28 except requests.exceptions.HTTPError as err:
29     return None, f"API failure: {err}"
30 except requests.exceptions.RequestException as err:
31     return None, f"API failure: {err}"
32
33 if not response.text or not response.text.strip():
34     return None, "API failure: Empty response from server."
35
36 try:
37     data = response.json()
38 except ValueError:
39     return None, "JSON decoding error: Could not parse API response."
40
41 if not isinstance(data, dict):
42     return None, "JSON decoding error: Unexpected response format."
43
44 if data.get("status") == "fail":
45     message = data.get("message", "Lookup failed.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\boundncher' '55123' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter IP address: 8.8.8.8

Geolocation Details
-----
Country : United States
City : Ashburn
ISP : Google LLC

PS C:\Users\shash\student_conversion>
```

AAC A 18.4.py X

C:\Users\shash> student_conversion > AAC A 18.4.py > ...

```
18 def fetch_geolocation(ip_value):
44     if data.get("status") == "fail":
45         message = data.get("message", "Lookup failed.")
46         return None, f"API failure: {message}"
47
48     country = data.get("country")
49     city = data.get("city")
50     isp = data.get("isp")
51
52     missing = []
53     if not country:
54         missing.append("country")
55     if not city:
56         missing.append("city")
57     if not isp:
58         missing.append("isp")
59
60     if missing:
61         return None, "API failure: Missing fields in response: " + ", ".join(missing)
62
63     return {
64         "country": country,
65         "city": city,
66         "isp": isp,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaco
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs
ncher' '55123' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter IP address: 8.8.8.8
```

Geolocation Details

```
-----
Country : United States
City    : Ashburn
ISP     : Google LLC
```

```
AAC A 18.4.py X
C: > Users > shash > student_conversion > AAC A 18.4.py > ...

68
69
70 def main():
71     ip_value = input("Enter IP address: ").strip()
72
73     if not is_valid_ip(ip_value):
74         print("Invalid IP format: Please enter a valid IPv4 or IPv6 address.")
75         return
76
77     location, error = fetch_geolocation(ip_value)
78
79     if error:
80         print(error)
81         return
82
83     print("\nGeolocation Details")
84     print("-" * 40)
85     print(f"Country : {location['country']}")
86     print(f"City    : {location['city']}")
87     print(f"ISP     : {location['isp']}")
88
89 if __name__ == "__main__":
90     main()

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion'; & 'c:\User
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64
ncher' '55123' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter IP address: 8.8.8.8

Geolocation Details
-----
Country : United States
City    : Ashburn
ISP     : Google LLC
```

Task 5 – Real-Time Application: Payment Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a sandbox payment API.

Task

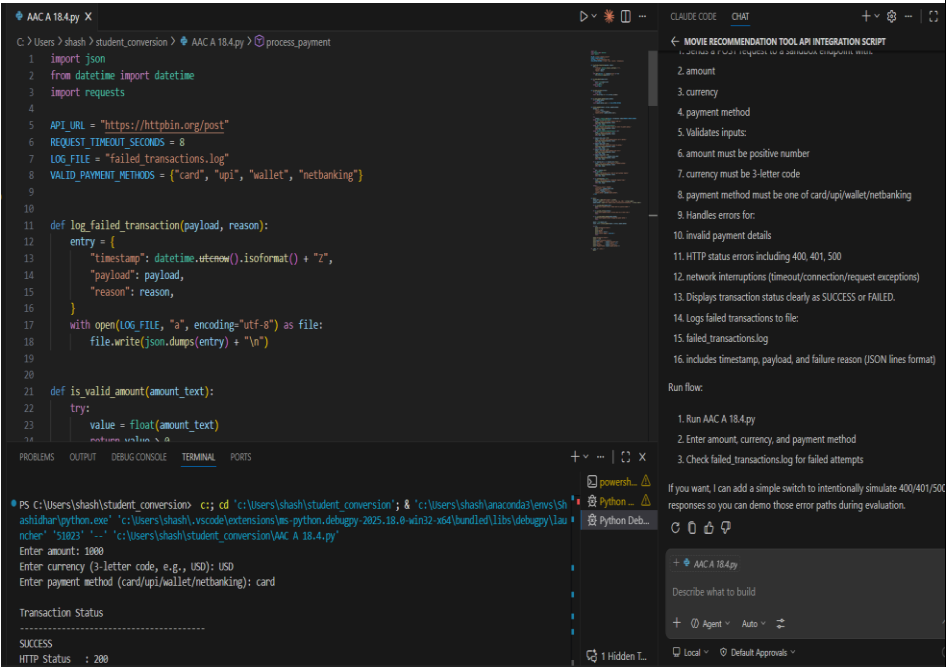
Use AI to generate a Python script that simulates payment processing.

Instructions

- Send POST request with:
 - Amount
 - Currency
 - Payment method
- Validate input values.
- Handle:
 - Invalid payment details
 - HTTP status errors (400, 401, 500)
 - Network interruptions
- Display transaction status clearly.
- Log failed transactions to a file.

Expected Output

- Script processes simulated payment.
- Errors handled gracefully.
- Failed requests logged properly.



AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > process_payment

```
28
29 def is_valid_currency(currency):
30     if not currency:
31         return False
32     return len(currency) == 3 and currency.isalpha()
33
34
35 def is_valid_payment_method(payment_method):
36     if not payment_method:
37         return False
38     return payment_method.lower() in VALID_PAYMENT_METHODS
39
40
41 def process_payment(amount, currency, payment_method):
42     payload = {
43         "amount": amount,
44         "currency": currency.upper(),
45         "payment_method": payment_method.lower(),
46     }
47
48     try:
49         response = requests.post(API_URL, json=payload, timeout=REQUEST_TIMEOUT_SECONDS)
50     except requests.exceptions.Timeout:
51         reason = "Network interruption: Request timed out."
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c:: cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anac
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs
ncher' '51023' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter amount: 1000
```

```
-----
SUCCESS
HTTP Status   : 200
Amount        : 1000.00
Currency      : USD
Payment Method: card
```

PS C:\Users\shash\student_conversion>

AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > process_payment

```
41 def process_payment(amount, currency, payment_method):
51     reason = "Network interruption: Request timed out."
52     log_failed_transaction(payload, reason)
53     return None, reason
54 except requests.exceptions.ConnectionError:
55     reason = "Network interruption: Unable to connect to payment gateway."
56     log_failed_transaction(payload, reason)
57     return None, reason
58 except requests.exceptions.RequestException as err:
59     reason = f"Network interruption: {err}"
60     log_failed_transaction(payload, reason)
61     return None, reason
62
63 if response.status_code == 400:
64     reason = "HTTP 400: Invalid payment details sent to gateway."
65     log_failed_transaction(payload, reason)
66     return None, reason
67 if response.status_code == 401:
68     reason = "HTTP 401: Unauthorized request to gateway."
69     log_failed_transaction(payload, reason)
70     return None, reason
71 if response.status_code == 500:
72     reason = "HTTP 500: Payment gateway server error."
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\ncher' '51023' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter amount: 1000
```

```
-----
SUCCESS
HTTP Status : 200
Amount : 1000.00
Currency : USD
Payment Method: card
```

AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > process_payment

```
41 def process_payment(amount, currency, payment_method):
74     return None, reason
75 if response.status_code >= 400:
76     reason = f"HTTP error: {response.status_code}"
77     log_failed_transaction(payload, reason)
78     return None, reason
79
80 if not response.text or not response.text.strip():
81     reason = "API failure: Empty response from payment gateway."
82     log_failed_transaction(payload, reason)
83     return None, reason
84
85 try:
86     data = response.json()
87 except ValueError:
88     reason = "JSON decoding error: Could not parse gateway response."
89     log_failed_transaction(payload, reason)
90     return None, reason
91
92 if not isinstance(data, dict):
93     reason = "JSON decoding error: Unexpected response format."
94     log_failed_transaction(payload, reason)
95     return None, reason
96
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-windows\bin\python.exe' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
```

Enter amount: 1000

SUCCESS

HTTP Status : 200
Amount : 1000.00
Currency : USD
Payment Method: card

PS C:\Users\shash\student_conversion> |

AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > process_payment

```
41 def process_payment(amount, currency, payment_method):
96
97     return {
98         "transaction_status": "SUCCESS",
99         "http_status": response.status_code,
100         "amount": payload["amount"],
101         "currency": payload["currency"],
102         "payment_method": payload["payment_method"],
103     }, None
104
105
106 def main():
107     amount_text = input("Enter amount: ").strip()
108     currency = input("Enter currency (3-letter code, e.g., USD): ").strip().upper()
109     payment_method = input("Enter payment method (card/upi/wallet/netbanking): ").strip().lower()
110
111     if not is_valid_amount(amount_text):
112         print("Invalid payment details: Amount must be a positive number.")
113         return
114
115     if not is_valid_currency(currency):
116         print("Invalid payment details: Currency must be a 3-letter code.")
117         return
118
119
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\environment\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51023' '---' 'c:\Users\shash\student_conversion\AAC A 18.4.py'

Enter amount: 1000

SUCCESS
HTTP Status : 200
Amount : 1000.00
Currency : USD
Payment Method: card

PS C:\Users\shash\student_conversion>

AAC A 18.4.py X

C: > Users > shash > student_conversion > AAC A 18.4.py > process_payment

```
106 def main():
122
123     amount = float(amount_text)
124     result, error = process_payment(amount, currency, payment_method)
125
126     if error:
127         print("\nTransaction Status")
128         print("-" * 40)
129         print("FAILED")
130         print(f"Reason: {error}")
131         print(f"Failure logged to {LOG_FILE}")
132         return
133
134     print("\nTransaction Status")
135     print("-" * 40)
136     print(result["transaction_status"])
137     print(f"HTTP Status   : {result['http_status']}")
138     print(f"Amount          : {result['amount']:.2f}")
139     print(f"Currency         : {result['currency']}")
140     print(f"Payment Method: {result['payment_method']}")
141
142 if __name__ == "__main__":
143     main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c::; cd 'c:\Users\shash\student_conversion';
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.
ncher' '51023' '--' 'c:\Users\shash\student_conversion\AAC A 18.4.py'
Enter amount: 1000
```

```
-----
SUCCESS
HTTP Status   : 200
Amount        : 1000.00
Currency      : USD
Payment Method: card
```

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots